

MISAKA

耐量子 BlockDAG Layer-1、Consensus-Native EVM、 単一整数正準クラスによる検証可能な LLM Work

The MISAKA Project

<https://github.com/MISAKA-BTC/misakas> · Explorer: <https://misakascan.com>

Whitepaper Version 3.0 – Research Specification – 2026-07-20

Abstract

要旨. MISAKA は *rusty-kaspa* を基盤とする PQ-only の耐量子 Layer-1 である。GHOSTDAG BlockDAG と UTXO ledger を維持しつつ、取引認証を ML-DSA-87、consensus-visible identity を 64-byte keyed BLAKE2b-512、UTXO commitment を additive lattice accumulator へ移行する。さらに、selected-parent execution に従属する EVM lane と、stake-confirmed canonical anchor および二次元 reorg gate を提供する DNS-style finality overlay を統合する。

Version 3 は PALW を再設計する。mint-grade LLM work は、ハードウェア別 pool や floating-point runtime class ではなく、単一の整数正準 compute set QW36-INT-W4A8-V1 に属する。NVIDIA、AMD、Apple の違いは consensus identity ではなく実装 telemetry である。全 backend は同一 artifact、tokenizer、chat template、固定小数点 ruleset、LUT、semantic schedule、checkpoint trace scheme を実行し、output、expert route、recurrent state、canonical execution root、compute units を bit exact に一致させる。

日常の local LLM 利用を mint へ接続する場合、requester/primary worker A は job-specific challenge-in-context の下で回答を生成し、結果を hiding commitment と escrow で先に固定する。commit 後の future beacon が、単一 bonded registry から replica B を credential 集約 bond に比例して選ぶ。B は A の結果を参照せず独立に full regeneration を行い、A の reveal 後に exact match を成立させる。全 leaf は public data availability の対象となり、別の future beacon で選ばれた auditor が一部を full replay する。quorum certificate と maturity を通過した leaf だけが one-shot ticket となる。block producer は通常の deterministic template 内で ticket を消費し、ML-DSA-87 block authorization で block preimage 全体を束縛する。full node は block path で model を再実行せず、commit、assignment、signature、bond、Merkle membership、certificate、maturity、ticket、authorization、nullifier、lane difficulty、compute cap を検証する。

Version 3 は「LLM の回答が真実である」ことや「特定 GPU が物理的に実行した」ことを主張しない。証明する対象は、登録済み正準 compute set の実行結果が複数の commit 後割当役割で一致し、監査可能な protocol history を経て one-shot work claim になったことである。correctness slashing は consensus TraceVM が divergent integer step を裁定できるまで無効とし、それ以前は署名不正、期限切れ、opening/DA 拒否、二重 commit 等の客観 fault だけを自動処理する。reviewed code snapshot では algo-4 acceptance は全 preset で無効、compute weight は zero、PALW payment は未 activation である。

Version 3 の主要変更

- PALW の 3 hardware pool / cross-vendor token-only matching を廃止し、単一 compute set と単一 bonded registry へ統合する。
- 正式 compute set を QW36-INT-W4A8-V1 のみに限定し、floating-point、Q4_K_M、graph fallback、Generic pricing を mint-ineligible とする。
- runtime_class_id を consensus match key から除外し、compute_set_id と非合意 implementation_id を分離する。
- self-order job を challenge-in-context、PCPB、escrow、future-beacon assignment、independent regeneration で first-class 化する。
- execution evidence を通常時の checkpoint chain と、紛争時だけ生成する op-level trace に分離する。
- mint path を all-open とし、private local inference は protocol 外・mint 不可とする。
- BatchCertificate を「署名の集合」ではなく、選出 auditor と sample plan を beacon から再導出し、full replay に束縛する object として定義する。
- block authorization を header preimage 全体へ束縛し、leaf/certificate/reward state を immutable にする。
- land / accept / weight を分離し、no-value testnet と economic activation の gate を明示する。

Status / Claim Boundary

本書は target protocol と reviewed code/ADR の統合仕様である。PQ base network と DNS overlay の実装状態、PALW code landing、QW36-INT runtime の project status、activation gate は同一ではない。PALW algo-4 block、provider reward、positive compute work は、本書に記載された gate を満たすまで有効化してはならない。

目次

1	はじめに	5
1.1	本書の貢献	5
1.2	本書の正本関係	5
2	Status、Scope、Claim Boundary	6
2.1	Normative vocabulary	6
2.2	実装状態	6
2.3	Claim boundary	7
3	予備知識と記法	7
4	System Overview	7
4.1	Design goal	7
4.2	Non-goal	8
5	耐量子 Cryptographic Core	8

5.1	ML-DSA-87 authorization	8
5.2	Hash64 identity	8
5.3	UTXO accumulator	9
6	Layered PoW と GHOSTDAG	9
6.1	Permanent hash floor	9
7	DNS Finality と Consensus Randomness	9
7.1	Bond と validator lifecycle	9
7.2	PALW 用 beacon 派生	9
8	Selected-Parent EVM と Execution Lane Roadmap	10
9	PALW Overview と Threat Model	10
9.1	Adversary capability	10
9.2	Security assumption	10
10	なぜ単一プールなのか	11
11	Canonical Compute Set: QW36-INT-W4A8-V1	11
11.1	compute_set_id	11
11.2	Consensus と telemetry の分離	12
12	厳密整数 Arithmetic	12
12.1	W4A8 domain	12
12.2	Rounding と requantization	12
12.3	Nonlinear operations	12
12.4	MoE と tie-breaking	12
12.5	Conformance	13
13	Self-Ordered Job と PCPB	13
13.1	Mint toggle	13
13.2	Job-specific freshness	13
13.3	PCPB flow	13
14	Single Bonded Provider Registry	13
15	Receipt、Checkpoint、DA、Leaf	14
15.1	Replica receipt	14
15.2	Checkpoint execution root	15
15.3	Receipt DA	15
15.4	LeafV2	15
15.5	Exact match	16

16	Objective Audit と BatchCertificate	16
16.1	Three-layer audit	16
16.2	Auditor vote と certificate	16
17	Mismatch、Dispute、TraceVM	17
18	Ticket Creation と algo-4 Block Construction	18
18.1	Block authorization	18
18.2	Construction order	18
18.3	Header admission	18
19	Component Work、Lane DAA、Fork Choice	18
20	Reward、Premium、Economic Safety	19
20.1	Current experimental split	19
20.2	No-show penalty	19
20.3	Collusion economics	19
21	Privacy と Public Auditability	19
22	P2P、DA、Denial of Service	20
23	Security Analysis	20
24	Implementation Status from Code and ADRs	21
24.1	Three activation levers	22
25	Testnet Ladder と Activation Gates	22
25.1	StopShip conditions	23
26	Governance、Versioning、Migration	23
27	Limitations と Open Questions	24
28	関連研究	24
29	結論	24
付録 A	Consensus Object Sketch	25
付録 B	Objective Verifier Pseudocode	26
付録 C	ADR / Code Traceability	26
付録 D	Parameter Classification	27

1 はじめに

MISAKA を動機づける圧力は三つある。第一は暗号である。classical elliptic-curve authorization と短い consensus identity は、十分な量子 adversary に対して長期 margin を維持しない。第二は構造である。linear chain は propagation delay を orphan pressure に変えるが、BlockDAG は concurrent honest block を保持し、GHOSTDAG により ordering へ変換する [1]。第三は計算資源である。local inference は日常的に大きな accelerator capacity を消費する一方、LLM execution をそのまま “proof of work” と呼ぶことはできない。再実行は高価で、input は機密になり得て、floating-point runtime は hardware-sensitive で、provider は Sybil identity を容易に増やせるからである [2]。

Version 1 は PQ UTXO consensus、Hash64 identity、layered PoW、selected-parent EVM を扱った。Version 2 は PALW audited-compute lane を導入したが、floating-point backend の不一致を hardware class と token-only matching で補う構造を残した。Version 3 はこの前提を取り除く。LLM work を単一整数正準 domain へ閉じ、security を hardware label ではなく、commit 後の役割抽選、exact semantic match、bond、escrow、public DA、future audit、maturity、bounded work accounting に置く。

1.1 本書の貢献

1. PQ-only UTXO ledger、GHOSTDAG ordering、DNS finality、selected-parent execution lane、PALW を一つの Base consensus 上で整理する。
2. PALW の唯一の初期 mint-grade compute set として QW36-INT-W4A8-V1 を定義する。
3. hardware pool を廃止し、単一 bonded provider registry から A、B、Auditor の役割を commit 後に割り当てる。
4. self-order job に対して job-specific challenge、hiding commit、escrow、post-commitment provider binding を定義する。
5. output token だけでなく、semantic schedule、expert route、state transition、checkpoint execution root、canonical CU の exact match を要求する。
6. 通常時の hash tax を checkpoint chain に抑え、紛争時だけ disputed segment を op-level に展開する。
7. BatchCertificate を future-randomized sample replay と stake/bond quorum に束縛する。
8. algo-4 block を permanent hash floor の上に置き、component-work cap

$$E = H + \min(C, 4H) \quad (1)$$

により、PALW stack が全て偽造されても hash work を無限に置換できない構造を維持する。

9. implementation status、StopShip condition、testnet ladder S0-S6、mainnet claim boundary を明示する。

1.2 本書の正本関係

本書は研究 whitepaper であり、wire encoding の正本ではない。優先順位は次のとおりである。

1. activated consensus code と static network parameters
2. Accepted ADR と versioned protocol specification
3. Proposed ADR (特に ADR-0040) と本 Whitepaper
4. explanatory implementation note

コードと本書が不一致の場合、activation を進めず、差分を ADR と test gate で解消する。

2 Status, Scope, Claim Boundary

2.1 Normative vocabulary

本書の **MUST**、**MUST NOT**、**SHOULD**、**MAY** は target rule を表す。 *Implemented* は reviewed tree に production path が存在すること、 *Landed/Inert* は code が存在するが acceptance または weight が閉じていること、 *Proposed* は ADR/design freeze、 *Research* は将来 dependency を表す。

2.2 実装状態

Component	Version 3 specification	Reviewed status
PQ UTXO authorization	ML-DSA-87 only、 PQ address/script、 classical signature path 無効	Base implementation track / public testnet 運用
Hash64 consensus identity	block/txid/merkle/UTXO/parent を 64-byte keyed BLAKE2b-512	Implemented per ADR-0008/0019
Layered PoW	Layer-1 tag + BLAKE2b-512 Layer-0 finalizer、 wide work accounting	Implemented/frozen base design
DNS finality	stake-confirmed canonical anchor + two-dimensional reorg gate + beacon	Implemented overlay; operational policy is network-specific
Selected-parent EVM	Base ordering に従属する EVM execution	Staged / feature-gated implementation
PALW core objects	manifest、 leaf、 certificate、 bond、 ticket、 authorization、 premium	Partially landed; several integrity remediations complete
QW36-INT runtime	W4A8 integer canonical inference	Project status: compute runtime completed; node registration/conformance binding remains
PCPB	A commit 後の B 選出、 B receipt、 A reveal	Design-only / not wired into live ticket verification
Real auditor replay	selected auditor が DA を取得し full replay	Not yet wired; current certificate path does not prove replay
Receipt DA	prompt/params/receipts/checkpoints の bounded availability	Root exists; transport/retention/refusal proof incomplete
Correctness TraceVM	disputed integer step の consensus adjudication	Research/design; correctness slash disabled
PALW acceptance	algo-4 block accepted by shipped preset	Disabled in reviewed branch

Component	Version 3 specification	Reviewed status
PALW compute weight	positive C in fork choice	Zero / not activated

2.3 Claim boundary

MISAKA の PQ claim は、Base UTXO authorization、consensus identity、PoW finalizer 等の明示された surface を対象とする。transport、compatibility EVM account、optional external service が全て PQ であるとは主張しない。

PALW が証明するのは、登録された compute set の deterministic execution claim が、commit 後割当、複製、監査、certificate、maturity、one-shot consumption を満たしたことである。次は主張しない。

- model output が現実世界で正しい、無害、有用であること。
- provider が自己申告した GPU vendor/型番を実際に使用したこと。
- TraceVM activation 前に、mismatch のどちら側が計算上不正かを consensus が判定できること。
- mint job の prompt が private であること。Version 3 の mint path は all-open である。
- provider pair が物理的に独立した二主体であること。Sybil control は bond share β の脅威仮定で扱う。

3 予備知識と記法

byte-string concatenation を $\|$ 、64-byte keyed BLAKE2b-512 を $H_d(x)$ 、canonical Merkle root を $\text{MerkleRoot}(S)$ と書く。全 consensus integer は fixed width、little-endian canonical encoding、明示的 rounding rule を持つ。overflow が仕様で許可されない場合、その input は invalid である。

H	permanent hash work component。
C	certified PALW compute work component。
E	fork choice に入る effective work。式 (1)。
W	snapshot における eligible matured bond の総額。
β	攻撃者が支配し得る eligible bond 比率。観測値ではなく脅威仮定。
m	A 以外に要求する replica 数。初期 wire v1 では $m = 1$ 、LeafV2 で可変化。
q	maturity window 内の監査確率または監査率下限。
T_m	leaf の maturity / challenge window。
Candidate leaf	A/B match と public commitment を持つが、まだ ticket でない work object。
BatchCertificate	batch、leaf root、auditor set、sample plan、vote、result を束縛する certificate。
Active ticket	certificate と maturity を通過し、一度だけ block に消費できる leaf。

4 System Overview

4.1 Design goal

- PQ-only Base ledger と GHOSTDAG throughput を維持する。
- EVM と AI compute を Base ordering に従属させ、独自 sequencer power を与えない。
- local LLM work を、block path から分離した audited work object として扱う。

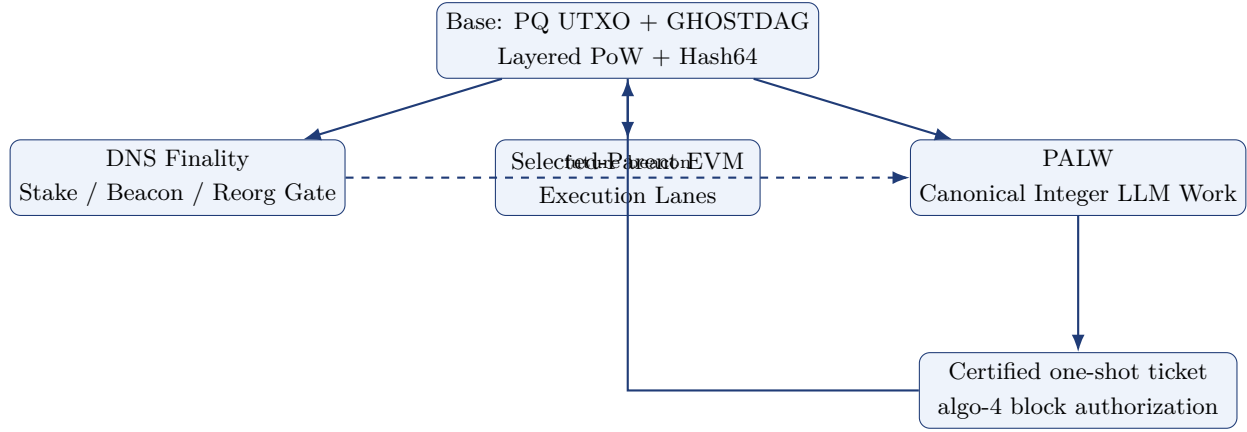


図 1: MISAKA Version 3 の層構造。PALW は Base ordering と hash floor に従属し、LLM が fork choice や transaction validity を直接決めることはない。

- full node の block validation を bounded hash/signature/state lookup/integer comparison に保つ。
- implementation、acceptance、economic weight を別 lever にし、未完成機構を zero-credit で land できるようにする。

4.2 Non-goal

- LLM を consensus oracle、truth oracle、transaction policy engine にすること。
- provider の hardware vendor を permissionless に証明すること。
- private prompt を複製しながら誰にも見せないこと。
- PALW が hash work を完全置換すること。
- committee 多数決だけで計算 correctness を確定すること。

5 耐量子 Cryptographic Core

5.1 ML-DSA-87 authorization

Base transaction authorization は ML-DSA-87 を用い、public key 2592 byte、signature 4627 byte の固定長を consensus mass と script policy に反映する。transaction signing、validator attestation、PALW auditor vote、PALW block authorization は別 domain/context を持ち、cross-protocol replay を禁止する。legacy secp256k1/Schnorr/ECDSA path は PQ-only Base consensus では表現不能とする。これは ADR-0019 と実装の ML-DSA-87 migration に従う [3]。

5.2 Hash64 identity

block hash、transaction id、merkle root、accepted-id root、UTXO commitment、parent reference は 64-byte keyed BLAKE2b-512 へ統一する。public claim は “512-bit commitment domain” と “256-bit quantum preimage margin” であり、256-bit quantum collision security は主張しない。BHT 型 collision bound は約 $2^{512/3}$ である。domain separation は keyed BLAKE2b の独立 key で実施する [4]。

5.3 UTXO accumulator

UTXO set commitment は additive lattice-style accumulator を使い、reorg、snapshot、trusted import で同じ state transition を再現する。これは transaction authorization と別の primitive であり、hash identity の幅と accumulator algebra を混同しない。

6 Layered PoW と GHOSTDAG

MISAKA の PoW は二層である。Layer 1 は `pow_algo_id` が選ぶ heavy tag、Layer 0 は network と algorithm を束縛した BLAKE2b-512 finalizer である。

$$L_1 = \text{AsicHardFn}_a(\text{prepow}, t, \text{bits}, n), \quad (2)$$

$$P = H_{\text{misaka-pow-v1}}(\text{network} \parallel a \parallel \text{prepow} \parallel t \parallel \text{bits} \parallel n \parallel |L_1| \parallel L_1). \quad (3)$$

P を 512-bit target と比較する。GHOSTDAG は concurrent block を blue cluster として ordering し、Base の canonical selected-parent view を作る [1]。PALW はこの ordering を変更せず、別 component C を追加するだけである。

6.1 Permanent hash floor

PALW block でも hash component H は永久に非零でなければならない。fork choice は $E = H + \min(C, 4H)$ を使い、compute stack の完全な破綻でも adversary の hash work amplification を最大 5 倍に制限する。 H と C は別 component として保持し、reorg、pruning、IBD で decomposition を失ってはならない。

7 DNS Finality と Consensus Randomness

DNS overlay は PoW/GHOSTDAG を置換しない。stake bond と ML-DSA-87 attestation により canonical lagged anchor を確認し、deep reorg がその prefix を外れる場合は common ancestor 以降の WorkScore と StakeScore の双方で canonical chain を上回ることを要求する。正確な public claim は “stake-confirmed canonical anchor + two-dimensional reorg gate” である。

7.1 Bond と validator lifecycle

bond は activation delay、unbonding delay、evidence window を持つ。equivocation evidence が有効な間に資本を逃がせないよう、unbonding period は reorg horizon と evidence window の和以上とする。reward eligibility と active set は candidate block の past-relative view から決定する。

7.2 PALW 用 beacon 派生

PALW は DNS/PALW commit-reveal beacon R_E を用途別に domain separation する。

$$R_{assign} = H_{\text{palw/assign/v1}}(R_E \parallel A_{commit} \parallel \text{snapshot_root}), \quad (4)$$

$$R_{audit} = H_{\text{palw/audit/v1}}(R_E \parallel \text{batch_id} \parallel \text{leaf_root}), \quad (5)$$

$$R_{conf} = H_{\text{palw/conformance/v1}}(R_E \parallel \text{credential_id} \parallel \text{round}), \quad (6)$$

$$R_{ticket} = H_{\text{palw/ticket/v1}}(R_E \parallel \text{leaf_id}). \quad (7)$$

各対象物は対応する beacon より規定 depth 前に fixed でなければならない。reveal withholding が無償なら seed bias が可能になるため、commit 後 no-reveal は客観 fault とし、degraded mode では新規 mint-grade assignment を停止する。

8 Selected-Parent EVM と Execution Lane Roadmap

selected-parent EVM は Base GHOSTDAG の sorted mergeset から input を抽出し、Base ordering に従って state を進める。EVM は独自 fork choice や sequencer order を持たない。native supply bridge は Base escrow を root of truth とし、deposit/withdraw は exactly-once system operation とする。

ADR-0023 は将来の Base + 3 execution lane を示す。Lane 1 は ML-DSA-87 native PQ-EVM、Lane 2 は Ethereum compatibility EVM、Lane 3 は proof-verified parallel execution の候補である。ただし PALW Version 3 はこの multi-lane activation を前提としない。共通不変条件は“Base だけが canonical order を決める”ことである。

9 PALW Overview と Threat Model

PALW は *Proof of Audited Local Work* として、block 生成前に実行された deterministic inference を candidate leaf へ変換し、複製、監査、certificate、maturity の後に one-shot ticket とする。

9.1 Adversary capability

adversary は次を行えると仮定する。

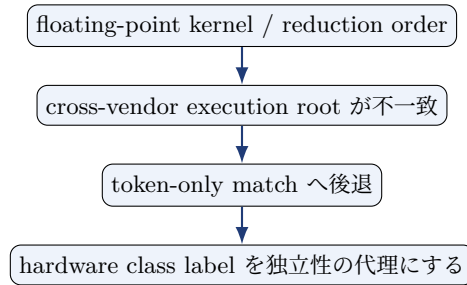
- 任意数の credential を作り、bond を複数 outpoint に分割する。
- A、B、Auditor の複数 role を同一運営者として支配する。
- hardware vendor、driver、実行時間等の telemetry を虚偽申告する。
- stale activation、cached output、precomputed trace、job replay を試みる。
- honest B を eclipse/DoS し、reroll を誘発する。
- certificate vote を withholding、偽造、replay、別 batch へ転用する。
- leaf、reward script、authorization、ticket を front-run または差し替える。
- DA object を withholding し、auditor の full replay を妨げる。

9.2 Security assumption

security は hardware label に依存しない。必要なのは、Base consensus と beacon の安全性、登録 set の canonical determinism、eligible bond の十分な slashable exposure、future-randomized role assignment、DA、監査率、maturity、activation cap である。攻撃者 bond share β は観測不能な脅威仮定であり、credential 集中度はその下界 telemetry にすぎない。

10 なぜ単一プールなのか

Version 2 の code は cross-vendor floating-point trace が異なるため、`runtime_class_id != other` を要求し、token output_commitment だけを比較した。これは次の因果関係である。



整数正準化により semantic execution root が backend 不変になれば、hardware label は不要である。同時に、permissionless network で vendor label を暗号的に強制できないため、label を security wall にすることもできない。したがって、全 provider は一つの registry に入り、役割だけを commit 後の beacon で分ける。

Design Invariant

単一 pool とは自己承認可能という意味ではない。全参加者が同じ登録集合に所属し、各ジョブの A、B、Auditor という役割だけを commit 後の予測不能な乱数で分離することを意味する。

11 Canonical Compute Set: QW36-INT-W4A8-V1

初期 mint-grade set は一つだけである。

```

QW36-INT-W4A8-V1
model: Qwen3.6-35B-A3B
weight: signed INT4
activation: signed INT8
accumulator: INT32 / specified INT64 paths
text-only, batch=1, independent regeneration
  
```

11.1 compute_set_id

$$compute_set_id = H(artifact_hash \parallel arithmetic_ruleset_hash \parallel LUT_roots) \quad (8)$$

$$\parallel tokenizer_hash \parallel chat_template_hash \quad (9)$$

$$\parallel sampler_policy \parallel PALW_prefix_format \quad (10)$$

$$\parallel shape_table \parallel overflow_budget_table \quad (11)$$

$$\parallel semantic_schedule_version \parallel checkpoint_policy \quad (12)$$

$$\parallel trace_scheme_version \parallel CU_ruleset_version). \quad (13)$$

artifact、LUT、tokenizer、chat template、overflow table のいずれかを変更した場合、同じ set ID を再利用してはならない。silent model replacement は禁止し、class-as-data により新しい set を並走させる。

11.2 Consensus と telemetry の分離

Consensus match key	Non-consensus telemetry
<code>compute_set_id</code>	CUDA / ROCm / Metal
artifact / ruleset / LUT root	GPU model / driver
output / schedule / execution root	kernel ID / tile / threadgroup
expert route / state root / CU	elapsed time / compiler ISA

backend-specific `implementation_id` は observability と regression triage に使えるが、mint validity や replica exact match の入力には使えない。

12 厳密整数 Arithmetic

12.1 W4A8 domain

weight は $[-8, 7]$ 、activation は $[-127, 127]$ とし、 -128 は reserved invalid value とする。最悪積は $8 \times 127 = 1016$ である。各 GEMM site は overflow budget table に K と operand domain を持ち、

$$K \cdot 1016 < 2^{31} \quad (14)$$

を満たすことを manifest validation で確認する。QW36 の実際の contraction dimension はこの上限より十分小さい。wrap を正当性根拠にせず、任意順序の部分和が範囲内であることを証明する。saturating addition は非結合なので禁止する。

12.2 Rounding と requantization

全除算・right shift は round-to-nearest, ties-to-even(RNE)へ統一する。`acc * multiplier` は INT64 intermediate で行い、negative tie を含む test vector を固定する。vendor library の floating-point alpha/beta エピログは禁止し、raw INT32 accumulator を取得して自前整数 requantization を行う。

12.3 Nonlinear operations

Softmax、RMSNorm、RoPE、SiLU/SwiGLU、router normalization、recurrent state update は、固定 LUT、整数除算、integer sqrt 等の versioned pure function で定義する。runtime-generated table、vendor `exp/rsqrt`、fast-math、implicit contraction は禁止する。KV cache と recurrent state も canonical integer encoding で保存する。

12.4 MoE と tie-breaking

各 token / MoE layer について routed expert IDs、fixed-point router weights、shared expert execution を commit する。score tie は小さい expert ID、token argmax tie は小さい token ID を選ぶ。route ordering、expert merge、shared expert、gated-delta recurrent transition を semantic schedule の一級 operation とする。

12.5 Conformance

conformance は hardware proof ではなく実装適合性試験である。primitive、requant、LUT、single layer、full model、long prefill/decode、recurrent growth、tie boundary、reject vector を CPU reference と各 backend で bit exact 比較する。許容 mismatch は zero である。

13 Self-Ordered Job と PCPB

13.1 Mint toggle

Version 3 の mint path は all-open である。

Mode	Prompt availability	Mint
Open / PALW	B、selected auditor、dispute participant が T_m 中取得可能	Candidate
Private local	device から出さない	不可

13.2 Job-specific freshness

$$job_challenge = H(network_id \parallel epoch_beacon \parallel scheduler_job_id) \quad (15)$$

$$\parallel requester_credential \parallel request_commitment \parallel shape_id). \quad (16)$$

canonical input は fixed PALW prefix、fixed challenge encoding、user prompt の順である。同一 prompt でも job ID と beacon が異なれば activation と state が変わる。freshness window と nullifier により replay を禁止する。

13.3 PCPB flow

$$A_{commit} = H_{palw/self-order-commit/v1}(job_descriptor \parallel receipt_fields \parallel r_{blind}). \quad (17)$$

B receipt deadline は A reveal より前で、B receipt は A_{commit} を署名対象に含む。A が opening を拒否した場合は客観 fault、B no-show は reroll count と penalty を発生させる。A が B を DoS して無料で再抽選できないよう、no-show reroll も attempt fee と上限を消費する。job nullifier と A commit は reroll 中に変更できない。

Version 3 v1 の B は independent regeneration を行うため $c_B \approx c_A$ である。teacher-forced verification は別 verification_mode_id と別 CU ruleset を要する将来 version とする。

14 Single Bonded Provider Registry

```
pub struct PalwProviderRecordV1 {
    credential_id: Hash64,
    compute_set_id: Hash64,
    bonded_value: u128,
    bond_outpoints_root: Hash64,
    bond_activation_epoch: u64,
    unbonding_epoch: Option<u64>,
    conformance_valid_until: u64,
```

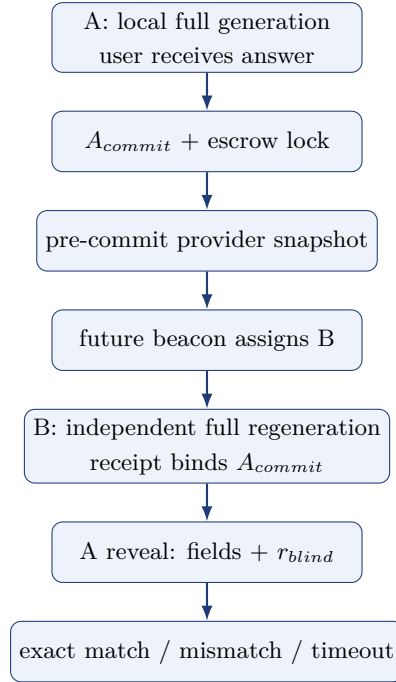


図 2: Post-commitment provider binding. A は B を知る前に結果と escrow を固定する。

```

reward_script: ScriptPublicKey,
registration_epoch: u64,
status: ProviderStatus,
}

```

bond は credential 単位に集約してから weighted sampling without replacement を行う。outpoint 単位抽選は、bond 分割で lottery ticket を増やすため禁止する。A 自身、既選択 B、unbonding/slashed、conformance expired、set mismatch を除外する。

自己ペア禁止は同一 credential を防ぐ衛生規則にすぎない。同一運営者が複数 credential を作る Sybil は残る。安全性は攻撃者 bond share β 、replica 数 m 、監査率 q 、slashable exposure、maturity によって評価する。

適合性 challenge は正準 output を返せることを確認するが、実際の hardware や全 job の honest execution を証明しない。不正防止の本体は本番 job の複製と監査である。

15 Receipt、Checkpoint、DA、Leaf

15.1 Replica receipt

ReceiptV3 は少なくとも次を束縛する。

```

ReplicaExecutionReceiptV3 {
  compute_set_id,
  scheduler_job_id,
  challenge_commitment,
  shape_id,
  output_commitment,
  operation_schedule_commitment,
  canonical_execution_root,
  expert_route_root,
  state_transition_root,
}

```

```

canonical_compute_units,
generated_token_count,
stop_reason,
provider_credential_id,
assignment_id,
A_commit,
signature_mlds87
}

```

15.2 Checkpoint execution root

通常時に全 operation output を hash すると prefill tax が大きい。したがって実行を fixed segment へ分け、増分 state commitment を作る。

```

SegmentCommitmentV1 {
  segment_index,
  phase,
  token_start,
  token_end,
  input_state_root,
  state_delta_root,
  output_state_root,
  output_token_prefix_root,
  expert_route_segment_root,
  semantic_counter_digest
}

```

$$canonical_execution_root = \text{MerkleRoot}(SegmentCommitment_0, \dots, SegmentCommitment_n). \quad (18)$$

state root は cache 全体の再 hash ではなく、canonical state writes による増分 chain とする。segment size、state serialization、chunk size は checkpoint_policy_id に含める。

15.3 Receipt DA

mint leaf は maturity window 中、prompt tokens、challenge tokens、decode params、A/B receipts、checkpoint list を content-addressed DA object として取得可能でなければならない。DA root だけを置き、request/response、retention、refusal proof が無い状態は evidence ではない。DA withholding は客観 fault とし、availability deadline を consensus view へ束縛する。

15.4 LeafV2

```

PalwReplicaLeafV2 {
  compute_set_id,
  scheduler_job_id,
  challenge_epoch,
  job_challenge_commitment,
  job_nullifier,
  A_commit,
}

```

```

provider_snapshot_root,
assignment_proof_root,
replica_count,
replica_set_root,
output_commitment,
operation_schedule_commitment,
canonical_execution_root,
expert_route_root,
state_transition_root,
canonical_compute_units,
reward_set_root,
receipt_da_root
}

```

leaf は manifest leaf root への Merkle membership を持ち、store は content-addressed write-once でなければならない。reward script は registry admission 時に PQ script/size を検査し、leaf commit 時点の snapshot へ固定する。payout 時に mutable current state を再参照してはならない。

15.5 Exact match

A と全 B は次を bit exact に一致させる。

- compute_set_id、shape、challenge、generated count、stop reason
- output commitment
- operation schedule commitment
- canonical execution root
- expert route root、state transition root
- canonical compute units

GPU vendor、driver、physical tile、kernel launch graph は比較しない。

16 Objective Audit と BatchCertificate

16.1 Three-layer audit

1. **全 leaf の cheap verification:** signature、bond、PCPB ordering、snapshot、assignment、challenge、nullifier、root match、DA availability、reward binding。
2. **sampled full replay:** future beacon が選んだ leaf を auditor が QW36-INT で再実行し、output / execution / route / state / CU を比較する。
3. **segment challenge:** sampled segment の checkpoint digest を再現し、必要なら dispute へ移す。

sample plan と auditor set は batch manifest/leaf root 固定後の future beacon から全 node が再導出する。certificate producer は vote aggregator にすぎず、auditor 選択や pass/fail を任意に決めない。

16.2 Auditor vote と certificate

```

BatchCertificateV2 {
    batch_id,

```

```

manifest_hash,
leaf_root,
provider_snapshot_root,
audit_beacon_epoch,
auditor_set_commitment,
sample_plan_root,
replay_result_root,
opening_result_root,
selected_stake,
responded_stake,
approving_stake,
votes_root
}

```

quorum は少なくとも次の三条件を同時に満たす。

$$responded_stake \geq \rho_{part} \cdot selected_stake, \quad (19)$$

$$approving_stake \geq \rho_{abs} \cdot selected_stake, \quad (20)$$

$$approving_stake \geq \rho_{rel} \cdot responded_stake. \quad (21)$$

沈黙した stake を単に denominator から消すだけでは withholding で quorum を縮められる。期限切れは再抽選 round と availability penalty を発生させる。

seeded certificate と quorum=1 self-audit は unit/demo/TX plumbing 専用で、compute weight と reward は zero とする。公開 testnet の mint-grade path は独立 auditor quorum を必須とする。

17 Mismatch、Dispute、TraceVM

mismatch が発生した場合、leaf と payout を凍結し、多数決だけで A/B のどちらが不正かを決めない。通常 leaf の checkpoint tree から最初の divergent segment を特定し、A/B はその segment だけを再実行して op-level tree を同時 commit する。両 commit 確定後に opening を行い、最初に異なる operation、logical tensor chunk、整数 step を二分探索する。この形は optimistic verification と interactive computation に近い [9, 10]。

最終 integer step を全 node が再計算するなら、それは consensus function である。したがって PalwIntegerTraceVmV1 が activation されるまで、計算 correctness に対する自動 slash は行わない。

TraceVM 前に自動処理できる客観 fault は次である。

- invalid signature / wrong domain
- deadline miss / no-show
- A opening refusal
- receipt DA refusal
- double commit / equivocation
- malformed proof / invalid Merkle path

correctness mismatch は leaf invalidation、reward zero、escrow による honest-side cost compensation、telemetry 保存までとする。

18 Ticket Creation と algo-4 Block Construction

batch lifecycle は概念的に

Registered → Committed → Certified → Active → Consumed/Expired

である。certificate と maturity を通過した leaf は、future beacon と global lane target により one-shot ticket lottery を行う。ticket nullifier は selected-parent-relative state で一度だけ消費される。

18.1 Block authorization

algo-4 block は ordinary PoW header ではなく ticket authority の ML-DSA-87 authorization を持つ。authorization は手書き field allowlist ではなく、canonical header preimage writer を再利用し、循環を避ける二置換だけを行う。

- `palw_authorization_hash = 0`
- transaction merkle root は authorization transaction を除いた authed root

これにより parents、coinbase、all header commitments、PALW fields、future field additions が自動的に束縛される。authorization transaction は canonical shape、block 内 exactly one、fixed position、no inputs/outputs、zero lock/gas/mass、strict payload round-trip を要求する。

18.2 Construction order

1. ticket winner、parents、coinbase、ordinary transaction set を固定する。
2. authorization tx を除く Merkle root と candidate header を固定する。
3. ticket authority が canonical authorization preimage を署名する。
4. canonical authorization tx を追加し final Merkle root を計算する。
5. 以後 header field を変更せず submit する。

full node は model を再実行せず、active batch、certificate、leaf membership、ticket ownership、target、authorization、nullifier、component headroom を検証する。

18.3 Header admission

algo-4 は Layer-0 PoW を免除するため、body 前に無制限 header を受けると DAG/store DoS になる。reviewed code では `palw_algo4_accept=false` がこれを閉じている。public testnet 前に GHOSTDAG-visible lane rate limit と compact header witness 等の deterministic admission cost を実装しなければならない。peer-local rate limit だけでは Sybil peer に弱い。

19 Component Work、Lane DAA、Fork Choice

H は ordinary hash work、 C は certified compute work である。effective work は式 (1)。compute cap は block 単位の部分適用ではなく、candidate increment 全体を受理または zero-credit とする all-or-nothing rule が望ましい。PALW lane difficulty は hash lane と別の過去 window から導出するが、hardware pool 別 target は持たない。

同一 leaf、job、ticket の重複 credit を拒否する。duplicate work、halted beacon、unbacked collateral は provider、validator、inclusion の全 payment axis で zero とし、別 actor へ迂回させない。

pruning、IBD、crash recovery は H/C decomposition、active batch、certificate、nullifier、beacon mode を復元しなければならない。external snapshot が effective work E だけを持ち H/C を失うことは許されない。

20 Reward、Premium、Economic Safety

20.1 Current experimental split

reviewed code の PALW lane experimental split は、subsidy base の 77% を provider role、15% を DNS validator、8% を worker-inclusion pool へ配分する。transaction-fee worker share は block assembler へ行く。これは mainnet promise ではなく、activation calibration 対象である。

provider base は replica premium π により

$$\sigma_A = \frac{1}{1+m\pi}, \quad \sigma_{B_i} = \frac{\pi}{1+m\pi} \quad (22)$$

と分ける。 $\pi = 1$ は $(1+m)$ 者の均等配分で、 $m = 1$ では従来の 50/50 を byte-identical に再現する。 π は DAA window から導出される bounded controller で、leaf commit window の値を frozen する。controller は activation 前に中立値で inert とし、パラメータは S-phase measurement 後に固定する。

20.2 No-show penalty

B no-show penalty は B bond から徴収し、burn または audit budget へ送る。A へ直接渡すと A が honest B を eclipse して利益を得るため禁止する。A の remedy は reroll と別建ての delay/attempt settlement である。repeat offence は bounded geometric escalation を持つ。

20.3 Collusion economics

攻撃者 bond share を β とすると、 m replica を全て引く確率は概ね β^m である。reroll が毎回 cost を持つ場合、Sybil B を全て引くまでの期待 cost は急増する。 β は on-chain で正確に観測できないため governance threat assumption として明示し、credential concentration、reroll、mismatch、no-show、slash telemetry は安全側の縮退 trigger に使う。

監査 wall は概念的に

$$q \cdot P(\text{attribution succeeds}) \cdot S_{\text{effective}} > V_{\text{fraud}} \quad (23)$$

を要求する。TraceVM 前は attribution probability が低く、correctness slash の $S_{\text{effective}}$ は zero とみなす。ゆえに高額 reward や positive fork-choice weight を有効にしてはならない。

21 Privacy と Public Auditability

Version 3 は content-hiding proof type、TEE-based validity、witness-hiding argument を mint path から除外する。mint-grade job は anyone-verifiable な open computation であり、少なくとも B、selected auditor、dispute participant が prompt と parameters を T_m 中取得できる。

private local inference は product feature として許されるが mint しない。TEE は将来 privacy curtain として利用可能でも、validity、work accounting、compute set identity へ寄与させない。hardware attestation failure が chain validity へ波及しない境界を維持する。

open prompt は model execution の再現性を高めるが、output の意味的 truth を保証しない。また、prompt/response が public であることから、秘密情報、未公開 source、credential、個人情報を mint job へ投入してはならない。

22 P2P、DA、Denial of Service

shared testnet には次の service が必要である。

- provider registration / conformance service
- replica dispatch / receipt submission
- DA publish / fetch / retention
- auditor assignment / replay / vote relay
- timeout / reroll / idempotent restart recovery

全 message は bounded vector、payload size cap、per-peer と global rate limit、content ID、expiry を持つ。DA object は block body と別に大きくなり得るため、root commitment と availability policy を分ける。malformed artifact、oversized checkpoint、duplicate receipt、orphan assignment を早期 reject する。

algo-4 header anti-spam は public no-value testnet の StopShip gate である。shared alpha/beta では archival と peer allowlist を強制し、public stage でのみ allowlist を外す。

23 Security Analysis

Attack	Mechanism	Remaining condition / limitation
Cached/replayed job	job-specific challenge、freshness、nullifier	challenge encoding と global/epoch state が consensus-wired であること
A fabricates result	post-commit B full regeneration、audit replay	attacker controls all selected roles with probability tied to β
B copies A	hiding commitment、B deadline before A reveal	receipt must bind A_{commit} and assignment
Bond split Sybil	credential aggregation + weighted sampling without replacement	multi-credential common control remains in β
Free reroll grinding	attempt fee、reroll cap、fixed A commit/nullifier	B submission path must resist A/relay eclipse
Fake certificate	ML-DSA-87 votes、active bond、quorum、rederived auditor set/sample plan	selected-set and sample rederivation must be live
Leaf substitution	Merkle membership、content-addressed write-once store	reward reads accepted immutable snapshot
Block front-running	total header-preimage authorization	miner constructor and validator must be byte-identical

Attack	Mechanism	Remaining condition / limitation
Ticket grinding	object fixed before future beacon, one-shot nullifier	beacon withholding/grinding bounded by base economics
DA withholding	availability deadline, objective fault penalty	DA transport and refusal proof required
Hardware label fraud	hardware removed from consensus predicate	no claim of physical vendor diversity
Runtime drift	compute set hash, conformance vectors, reject vectors	artifact/LUT/version governance required
Integer overflow	typed W4/A8, overflow budget, checked oracle	every model op/shape must be covered
Auditor majority lies	certificate only admits; TraceVM adjudicates correctness	before TraceVM, correctness slash disabled
Header flood	deterministic algo-4 admission cost/rate limit	unresolved gate before public acceptance

24 Implementation Status from Code and ADRs

Version 3 は ADR-0007/0008/0009/0016–0019/0020/0023/0040 と code snapshot `misakas-feat-dns-dormancy-core-on-main-4dcdee8` を主要入力とする。コード各所と ADR-0040 は ADR-0039 を参照するが、ADR-0039 本文は reviewed archive に含まれていなかったため、本書はその内容をコードコメント、ADR-0040 の訂正記録、および実装 surface からのみ扱う。重要な現況は次である。

Item	Status	Evidence / implication
ML-DSA-87 / Hash64 / PQ-only Base	Implemented	Base consensus and address/script path
DNS bond / attestation / finality	Implemented/staged	public testnet operation; network policy differs
PALW leaf membership / write-once	Remediated	content-addressed storage and membership proof path
Certificate signature / active bond / quorum	Remediated, partial	declared set is checked; selected-set/sample rederivation remains
AUTH-02 block authorization	Remediated	total canonical header preimage binding + canonical auth tx
Dynamic premium π	Implemented but inert	neutral default, DAA-window integer controller
Proof type privacy classes	Removed	PALW mint is all-open

Item	Status	Evidence / implication
PCPB helper	Exists but intentionally unwired	current enum carries caller-asserted booleans; LeafV2 atomic slice required
Provider/auditor selection	Legacy unweighted helper only	credential bond-weighted sampler has no live path
Receipt DA	Root field only	object/transport/retention/refusal proof absent
Real auditor replay	Not implemented	current producer can consume caller-provided pass result
ComputeSet registration	Type exists / live caller missing	S0-f and registry TX/store required
QW36-INT node integration	Pending in reviewed tree	runtime completion does not equal consensus registration
Header anti-spam	Not implemented	algo-4 acceptance remains false
TraceVM / correctness slashing	Design-only	only objective faults may be automated

24.1 Three activation levers

Land	code、types、tests を main tree へ入れる。consensus behaviour は変えない。
Accept	static network preset が algo-4 object/block を受理する。
Weight	certified compute C が positive fork-choice work と economic payout を持つ。

Version 3 はこの順序を逆転させてはならない。runtime CLI flag で consensus accept rule を operator ごとに変えることも禁止する。

25 Testnet Ladder と Activation Gates

Stage	Scope	Exit condition
S0-a	INT4 unpack、INT8 GEMM、INT32 accumulate、RNE	CPU reference と全 backend bit exact、reject vector pass
S0-b	requant、INT64/hierarchical reduction	block width/order independence
S0-c	Softmax/RMSNorm/SiLU/RoPE LUT	byte roots 一致、runtime table generation なし
S0-d	single layer、MoE route/shared expert、recurrent state	layer/state/route root 一致
S0-e	full model、long prefill、256–1024 decode、tie boundary	full root mismatch zero

Stage	Scope	Exit condition
S0-f	artifact/vector commitment と ComputeSet registration	weight_factor_bps=0 で set が chain state に存在
S1	single registry、bond maturity、conformance、weighted sampler	bond split が抽選確率を変えない
S2	PCPB multi-node、timeout/reroll/escrow/nullifier	A commit 前 snapshot、B independent execution、free reroll なし
S3	real auditor selection/replay/quorum/certificate	forged/zero-stake/unselected auditor 全拒否
S4	dispute shadow mode	divergent segment/chunk 特定、escrow 補償、correctness slash off
S5	no-value public testnet	compute weight 0、main subsidy 影響なし、reorg/crash/IBD/DoS soak
S6	capped activation	small cap、fixed m 、high q 、long maturity、automatic halt

25.1 StopShip conditions

次のいずれかが未達なら algo-4 acceptance または positive weight を開かない。

- QW36-INT artifact hash、overflow table、LUT root、vector commitment が実値で固定されない。
- PCPB、credential-weighted assignment、global/epoch nullifier、DA、real auditor replay が live verifier に接続されない。
- certificate selected-set/sample-plan を beacon から独立再導出できない。
- algo-4 header admission に deterministic cost boundary が無い。
- reward が accepted immutable leaf/certificate snapshot に束縛されない。
- reorg、pruning、IBD、crash recovery で同じ PALW view と H/C component を再構築できない。
- TraceVM 無しで correctness slash や高額 economic exposure を有効化しようとする。

26 Governance、Versioning、Migration

hardware class は consensus class ではない。複数 class が存在するのは compute specification version の移行期だけである。

```
QW36-INT-W4A8-V1  ->  QW36-INT-W4A8-V2
(old set active)    (new artifact/rules/LUT/new set_id)
```

新 set は zero weight で登録し、S0 conformance、S1 registry、S2/S3 interoperability を通過してから weight を移す。artifact を同じ set ID のまま差し替えてはならない。deactivation は non-retroactive とし、既に有効だった receipt/ticket の historical verification を維持する。

LeafV2、ReceiptV3、ML-DSA-87 signature unification、challenge binding、compute set field、HeaderV4 等の wire 変更は、既存 public network へ小刻みに混ぜず、new network ID / coordinated reset / re-genesis を一度に行う方が安全である。static preset と machine-enforced gate を用い、CLI override で validator rule を分岐させない。

27 Limitations と Open Questions

1. **Checkpoint policy**: prefill/decode segment size と hash overhead の最終値は S0 benchmark 後に凍結する。
2. **TraceVM**: instruction set、state commitment、single-step cost、timeout、witness format が未確定。
3. β_{max} 、 m 、 q : β は観測不能な threat assumption で、初期値と縮退曲線は governance decision を要する。
4. **DA retention**: maturity、reorg horizon、audit latency、storage cost の trade-off を実測する必要がある。
5. **Header anti-spam**: lane rate limit と compact witness の閾値を adversarial benchmark で決める必要がある。
6. **35B-A3B overflow table**: 全 op $\times$ max shape の表と machine-verifiable proof が必要である。
7. **Quality**: W4A8 quality は protocol validity と別であり、公開 benchmark を activation governance に使うか決める必要がある。
8. **Hardware independence**: bit exact output は execution implementation の一致を示すが、物理 device identity を示さない。
9. **Semantic truth**: replica/auditor が同じ誤答を再現しても protocol は成立する。PALW は truth oracle ではない。

28 関連研究

MISAKA の DAG ordering は GHOSTDAG[1]、Sybil threat は unconstrained identity の古典的問題 [2]、PQ signature は NIST ML-DSA[3]、Hash64 は BLAKE2[4] に基づく。DNS overlay は distributed randomness と stake-assisted reorg gate に関係する [6, 7]。selected-parent EVM と将来 parallel lane は parallel execution research[8] を参照する。

PALW dispute は optimistic verification[9, 10]、matrix check は Freivalds 法 [11]、resource-backed proof system の対照として Filecoin[12] に関連する。ただし Version 3 は succinct proof や ZKML を current mint path の前提にせず、open deterministic recomputation と economic exposure を採る。

29 結論

MISAKA Version 3 は PQ-only Base consensus、Hash64 identity、layered PoW、GHOSTDAG ordering、DNS finality、selected-parent execution lane を維持し、PALW を単一整数正準 compute domain へ再構成する。hardware pool は廃止され、全 provider は一つの bonded registry に入り、A、B、Auditor の役割だけが commit 後の future beacon で分けられる。

日常の local inference は、open mint toggle を選んだ場合にのみ PALW job になる。A は job-specific challenge の下で回答を得た後、結果を hiding commit と escrow で固定する。B は独立に full regeneration を行い、全 semantic roots を exact match する。全 leaf は audit draw 前に public DA へ固定され、selected auditor が full replay し、certificate と maturity を通過したものだけが one-shot ticket になる。block producer は ticket を ordinary deterministic template へ取り込み、ML-DSA-87 authorization が block preimage 全体を束縛する。

中心的な security choice は、hardware label や operator independence を識別できると装わないことである。安全性は credential-aggregated bond、future-randomized role assignment、escrow、maturity、public replay、component-work cap、automatic halt に置く。TraceVM が完成するまで correctness slash を封印し、客観 fault のみを処理する。全 gate が満たされるまで algo-4 acceptance と positive compute weight は zero である。

Design Invariant

単一 pool とは自己承認可能という意味ではない。全参加者が同じ登録集合に所属し、各ジョブの A、B、Auditor という役割だけを commit 後の予測不能な乱数で分離することを意味する。

付録 A Consensus Object Sketch

以下は explanatory sketch であり、wire は fixed width、canonical serialization、bounded vector、version domain を必要とする。

```

PalwComputeSetRecordV1 {
  set_id, model_artifact_hash, arithmetic_ruleset_hash,
  tokenizer_hash, chat_template_hash, lut_roots,
  overflow_budget_root, shape_table_root,
  schedule_schema_id, checkpoint_policy_id,
  trace_scheme_id, cu_ruleset_id,
  vector_commitment, weight_factor_bps,
  activation_daa, deactivation_daa
}

PalwACommitV2 {
  scheduler_job_id, compute_set_id, challenge_commitment,
  request_commitment, shape_id, a_commit,
  escrow_outpoint, registration_epoch, signature
}

PalwReplicaAssignmentV2 {
  scheduler_job_id, a_commit, snapshot_root,
  beacon_epoch, replica_index, provider_credential_id,
  weighted_draw_proof
}

ReplicaExecutionReceiptV3 {
  assignment_id, a_commit, provider_credential_id,
  compute_set_id, output_commitment,
  operation_schedule_commitment,
  canonical_execution_root, expert_route_root,
  state_transition_root, canonical_compute_units,
  generated_token_count, stop_reason, signature
}

PalwReplicaLeafV2 {
  compute_set_id, scheduler_job_id, challenge_epoch,
  job_challenge_commitment, job_nullifier, a_commit,
  provider_snapshot_root, assignment_proof_root,
  replica_count, replica_set_root,
  output_commitment, operation_schedule_commitment,
  canonical_execution_root, expert_route_root,
  state_transition_root, canonical_compute_units,
  reward_set_root, receipt_da_root
}

```

```
BatchCertificateV2 {
  batch_id, manifest_hash, leaf_root,
  provider_snapshot_root, audit_beacon_epoch,
  auditor_set_commitment, sample_plan_root,
  replay_result_root, opening_result_root,
  selected_stake, responded_stake, approving_stake,
  votes_root
}
```

付録 B Objective Verifier Pseudocode

```
verify_palw_block(block, selected_parent):
  view = resolve_palw_view(selected_parent)
  require view.static_params.palw_algo4_accept
  require cheap_header_admission(block.header, view)

  binding = resolve_active_ticket(block.header, view)
  verify_manifest_certificate_leaf_binding(binding)
  verify_leaf_membership_and_write_once_identity(binding)
  verify_compute_set_active(binding.compute_set_id, view)
  verify_challenge_freshness(binding)
  verify_pcpb_assignment_and_receipts(binding, view)
  verify_certificate_selected_set_and_sample_plan(binding, view)
  verify_maturity_and_da(binding, view)
  verify_provider_collateral_and_reward_snapshot(binding, view)

  require derive_ticket_target(binding, view) == block.header.bits
  require derive_ticket_nullifier(binding) == block.header.ticket_nullifier
  require not view.ticket_nullifiers.contains(block.header.ticket_nullifier)
  require verify_total_header_authorization(block, binding.ticket_authority)

  delta_c = derive_certified_compute_increment(binding, view)
  require delta_c <= compute_headroom(view.H, view.C, cap=4)

  result = ghostdag_with_component_work(block, delta_c)
  require result.H == block.header.hash_work
  require result.C == block.header.compute_work
  require result.E == block.header.blue_work
  commit_atomically(result)
```

付録 C ADR / Code Traceability

Whitepaper area	Governing records	Representative code surface
Layered PoW	ADR-0007	consensus/core/src/pow_layer0.rs, header/PoW validation
Hash64	ADR-0008	crypto/hashes, block/tx/merkle types
DNS finality	ADR-0009, ADR-0010–0018	DNS stores, validator tools, reorg gate

Whitepaper area	Governing records	Representative code surface
ML-DSA-87	ADR-0019	<code>crypto/txscript</code> , signature domains
Selected-parent EVM	ADR-0020	EVM process/executor/RPC
Execution lane roadmap	ADR-0023	multi-lane design and future header roots
PALW base design	ADR-0039*	<code>consensus/core/src/palw.rs</code> , miner components
Single-pool integer remediation	ADR-0040	compute set, PCPB, gates S0–S6
PALW rewards	ADR-0039 §17*, ADR-0040 §16'	<code>coinbase.rs</code> , <code>palw_premium.rs</code>
PALW authorization	ADR-0040 AUTH-02	<code>mil/miner/src/authorization.rs</code> , block validation
Implementation status	<code>docs/palw-tshared-progress-2026-07-20.md</code>	presets, DB version, tests, PR state

* ADR-0039 is referenced throughout the code and by ADR-0040, but its source file was not present in the reviewed archive. This row is grounded in code references and ADR-0040's corrective record.

付録 D Parameter Classification

Parameter	Classification	Version 3 rule
Hash floor	Normative	permanent/nonzero
Global compute cap	Normative	$C \leq 4H$ contribution cap
Initial compute set	Normative target	QW36-INT-W4A8-V1 only
Hardware class	Non-consensus	telemetry only
Replica mode v1	Normative	independent full regeneration
Replica count m	Provisional/governed	v1 wire $m = 1$; LeafV2 permits bounded m
Audit probability q	Hard-fork/governed	conservative lower bound; calibrated before reward
Threat share β_{max}	Explicit assumption	not derived on-chain
Checkpoint segment size	Provisional	S0 benchmark then new policy ID
Reward premium π	Inert controller	neutral=1; bounded DAA-window updates after activation
Correctness slash	Disabled until TraceVM	objective faults only beforehand
Private job	Product/off-protocol	no mint
TEE	Optional privacy only	no validity or credit uplift

Parameter	Classification	Version 3 rule
Strong proof	Research	separate proof type and accounting

参考文献

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar, “PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus,” Cryptology ePrint Archive 2018/104, 2018.
- [2] J. R. Douceur, “The Sybil Attack,” International Workshop on Peer-to-Peer Systems, 2002.
- [3] National Institute of Standards and Technology, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024.
- [4] M.-J. O. Saarinen and J.-P. Aumasson, *The BLAKE2 Cryptographic Hash and Message Authentication Code*, RFC 7693, 2015.
- [5] K. Lewi et al., “Securing Update Propagation with Homomorphic Hashing,” Cryptology ePrint Archive 2019/227, 2019.
- [6] E. Syta et al., “Scalable Bias-Resistant Distributed Randomness,” IEEE Symposium on Security and Privacy, 2017.
- [7] League of Entropy, *drand: Distributed Randomness Beacon*, 2020.
- [8] R. Gelashvili et al., “Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing,” arXiv:2203.06871, 2022.
- [9] J. Teutsch and C. Reitwiessner, “A Scalable Verification Solution for Blockchains,” arXiv:1908.04756, 2019.
- [10] H. Kalodner et al., “Arbitrum: Scalable, Private Smart Contracts,” USENIX Security Symposium, 2018.
- [11] R. Freivalds, “Probabilistic Machines Can Use Less Running Time,” IFIP Congress, 1979.
- [12] Protocol Labs, *Filecoin: A Decentralized Storage Network*, 2017.